

An Efficient Algorithm for a New Constrained LCS Problem

Daxin Zhu¹, Yingjie Wu², and Xiaodong Wang³(✉)

¹ Quanzhou Normal University, Quanzhou 362000, China

² Fuzhou University, Fuzhou 350002, China

³ Fujian University of Technology, Fuzhou 350108, China
wangxd135@139.com

Abstract. The solution to a generalized longest common subsequence problem is addressed in this paper. In this problem, two given sequences of their length s and t must be included as the subsequences of the two main sequences A and B , and the result subsequences must be the longest. If the lengths of the two main sequences A and B are n and m , and the lengths of their two constrained sequences are s and t respectively, the time complexity of the dynamic programming algorithm presented in this paper to solve the new generalized LCS problem must be $O(nmst)$.

Keywords: Generalized LCS problem · Similarity · Constrained sequences · Time complexity, Dynamic programming algorithm

1 Introduction

LCS problem (the longest common subsequence) can be declared as follows. For the given two sequences A and B , we want to find a common subsequence of the main sequences A and B of the longest length.

This problem is a well-known problem for measuring the similarity of two sequences. It seems to have a lot of applications in various apparently unrelated fields, such as computer science, mathematics, molecular biology, file comparison, pattern matching, speech recognition, gas chromatography [3, 4, 8, 9].

For some biological applications the new variants of the LCS problems are considered. In such problems, some constraints can be applied to the longest common subsequence problem, and the different kinds of variant of the longest common subsequence problem are commonly called the constrained LCS (CLCS) problem [2, 4, 6, 7, 10, 11]. Recently, a new kind of generalized forms of the constrained LCS problem, the generalized constrained LCS (GC-LCS) problem was proposed by Chen and Chao [1]. For the two input sequences A and B with their lengths m and n , respectively, and a constrained string U of length r , the generalized constrained LCS problem can be formulated as a set of four problems to find the longest common subsequence of A and B including/excluding U as a subsequence or a substring, respectively.

The purpose of this paper is to consider another more general constrained LCS problem called SEQ-IC-SEQ-IC-LCS, in which two constrained sequences with their lengths s and t must be included as the subsequences of the two main sequences A and B , and the length of the resulting subsequences must be maximal. A first efficient dynamic programming algorithm will be presented to solve this problem.

We will organize our paper as follows.

Our main result on the original presented dynamic programming algorithm to solve the SEQ-IC-SEQ-IC-LCS problem will be described in the following 3 sections.

In Sect. 2, we will discuss the preliminary knowledge to present our algorithm for the newborn SEQ-IC-SEQ-IC-LCS problem. In Sect. 3, the new dynamic programming algorithm to solve the SEQ-IC-SEQ-IC-LCS problem is presented, and its give time complexity $O(nmst)$ is given, where n and m are their lengths of the two main input sequences A and B , and the lengths of the two constrained sequences are s and t , respectively. In Sect. 4, we give some concluding comments.

2 Some Characterizations to the New SEQ-IC-SEQ-IC-LCS Problem

A sequence can be characterized as a string of characters over some alphabet Σ . A subsequence is obtained by deleting some characters(not necessarily contiguous), zero or more, from the original sequence A . A substring is a subsequence of A and its characters are successive within A .

For a given sequence $A = a_1a_2 \cdots a_n$ with length n , its i th character can be denoted as $a_i \in \Sigma$ for any $i = 1, \dots, n$. $A[i : j] = a_ia_{i+1} \cdots a_j$ is used to denote the substring of A from position i to j . The substring $A[i : j] = a_ia_{i+1} \cdots a_j$ is designated as a proper substring of A , if $i \neq 1$ or $j \neq n$. If $i = 1$ or $j = n$, then the substring $A[i : j] = a_ia_{i+1} \cdots a_j$ is referred to as a prefix of A or a suffix of A , respectively.

An appearance of sequence $A = a_1a_2 \cdots a_n$ in sequence $B = b_1b_2 \cdots b_m$, for any given sequences A and B , beginning from the position j is a strictly increasing indexes sequence $i_1, i_2, \dots, i_n$, such that $i_1 = j$, and $A = b_{i_1}, b_{i_2}, \dots, b_{i_n}$. A compact appearance of A in B beginning from the position j is its appearance with the smallest last index i_n .

For the two input sequences $A = a_1a_2 \cdots a_n$ and $B = b_1b_2 \cdots b_m$ with their lengths m and n , respectively, and the two constrained sequences $U = u_1u_2 \cdots u_s$ and $V = v_1v_2 \cdots v_t$ with their lengths s and t , the SEQ-IC-SEQ-IC-LCS problem is defined as finding a constrained longest common subsequence of A and B including U and V as its subsequences.

Definition 1. Let $L(i, j, k, r)$ be the set of all longest common subsequences of $A[1 : i]$ and $B[1 : j]$ such that for each $c \in L(i, j, k, r)$, c includes $U[1 : k]$ and $V[1 : r]$ as subsequences, where $0 \leq r \leq t$, and $1 \leq i \leq n, 1 \leq j \leq m, 0 \leq k \leq s$. The length of a longest common subsequence in $L(i, j, k, r)$ is denoted as $g(i, j, k, r)$.

To compute the longest common subsequences in $L(i, j, k, r)$, for any $1 \leq i \leq n, 1 \leq j \leq m, 0 \leq k \leq s$, and $0 \leq r \leq t$, we will use the following theorem to characterize the structure of the optimal solution based on the properties of the optimal solutions to their subproblems.

Theorem 1. *If $C[1 : l] = c_1, c_2, \dots, c_l \in L(i, j, k, r)$, then we can build the following conditions to the optimal solutions:*

1. *If $k > 0, r > 0, a_i = b_j = u_k = v_r$, then $c_l = a_i$ and $C[1 : l - 1] \in L(i - 1, j - 1, k - 1, r - 1)$.*
2. *If $k > 0, a_i = b_j = u_k$ and $(r = 0 \vee a_i \neq v_r)$, then $c_l = a_i$ and $C[1 : l - 1] \in L(i - 1, j - 1, k - 1, r)$.*
3. *If $r > 0, a_i = b_j = v_r$ and $(k = 0 \vee a_i \neq u_k)$, then $c_l = a_i$ and $C[1 : l - 1] \in L(i - 1, j - 1, k, r - 1)$.*
4. *If $a_i = b_j$ and $(k = 0 \vee a_i \neq u_k)$ and $(r = 0 \vee a_i \neq v_r)$, then $c_l = a_i$ and $C[1 : l - 1] \in L(i - 1, j - 1, k, r)$.*
5. *If $a_i \neq b_j$, then $c_l \neq a_i$ implies $C[1 : l] \in L(i - 1, j, k, r)$.*
6. *If $a_i \neq b_j$, then $c_l \neq b_j$ implies $C[1 : l] \in L(i, j - 1, k, r)$.*

Proof.

1. In this first case, we can add a_i to $C[1 : l - 1]$ such that $a_i = c_l$, and thus $C[1 : l - 1]$ must be an LCS of $A[1 : i - 1]$ and $B[1 : j - 1]$ including $U[1 : k - 1]$ and $V[1 : r - 1]$ as subsequences, i.e. $C[1 : l - 1] \in L(i - 1, j - 1, k - 1, r - 1)$.
2. In this second case, we have no constraints on V , and thus a_i can be added to $C[1 : l - 1]$ and we can conclude that $C[1 : l - 1]$ is a common subsequence of $A[1 : i - 1]$ and $B[1 : j - 1]$ including $U[1 : k - 1]$ as a subsequence and including $V[1 : r]$ as a subsequence. It can be seen further that $C[1 : l - 1]$ is an LCS of $A[1 : i - 1]$ and $B[1 : j - 1]$ including $U[1 : k - 1]$ as a subsequence and including $V[1 : r]$ as a subsequence. If there is a common subsequence a of $A[1 : i - 1]$ and $B[1 : j - 1]$, assumed by contradiction, including $U[1 : k - 1]$ as a subsequence and including $V[1 : r]$ as a subsequence, and its length must be greater than $l - 1$. Then we can concatenate a and a_i to form a common subsequence of $A[1 : i]$ and $B[1 : j]$ including $U[1 : k]$ and $V[1 : r]$ as subsequences, and its length will be greater than l . This will be a contradiction.
3. In this third case, we have no constraints on U , and thus a_i can be added to $C[1 : l - 1]$ and we can conclude that $C[1 : l - 1]$ is a common subsequence of $A[1 : i - 1]$ and $B[1 : j - 1]$ including $U[1 : k]$ as a subsequence and including $V[1 : r - 1]$ as a subsequence. It can be seen further that $C[1 : l - 1]$ is an LCS of $A[1 : i - 1]$ and $B[1 : j - 1]$ including $U[1 : k]$ as a subsequence and including $V[1 : r - 1]$ as a subsequence. If there is a common subsequence a of $A[1 : i - 1]$ and $B[1 : j - 1]$, assumed by contradiction, including $U[1 : k]$ as a subsequence and including $V[1 : r - 1]$ as a subsequence, and its length must be greater than $l - 1$. Then we can concatenate a and a_i to form a common subsequence of $A[1 : i]$ and $B[1 : j]$ including $U[1 : k]$ and $V[1 : r]$ as subsequences, and its length will be greater than l . This will be a contradiction.

4. In this fourth case, we have no constraints on U and V , and thus a_i can be added to $C[1 : l - 1]$ and we can conclude that $C[1 : l - 1]$ is a common subsequence of $A[1 : i - 1]$ and $B[1 : j - 1]$ including $U[1 : k]$ and $V[1 : r]$ as subsequences. It can be seen further that $C[1 : l - 1]$ is an LCS of $A[1 : i - 1]$ and $B[1 : j - 1]$ including $U[1 : k]$ and $V[1 : r]$ as subsequences. If there is a common subsequence a of $A[1 : i - 1]$ and $B[1 : j - 1]$, assumed by contradiction, including $U[1 : k]$ and $V[1 : r]$ as subsequences, and its length must be greater than $l - 1$. Then we can concatenate a and a_i to form a common subsequence of $A[1 : i]$ and $B[1 : j]$ including $U[1 : k]$ and $V[1 : r]$ as subsequences, and its length will be greater than l . This will be a contradiction.
5. Since $a_i \neq b_j$ and $c_l \neq a_i$, we can conclude that $C[1 : l]$ is a common subsequence of $A[1 : i - 1]$ and $B[1 : j]$ including $U[1 : k]$ and $V[1 : r]$ as subsequences. It is obvious that $C[1 : l]$ is also an LCS of $A[1 : i - 1]$ and $B[1 : j]$ including $U[1 : k]$ and $V[1 : r]$ as subsequences.
6. Since $a_i \neq b_j$ and $c_l \neq b_j$, we can conclude that $C[1 : l]$ is a common subsequence of $A[1 : i]$ and $B[1 : j - 1]$ including $U[1 : k]$ and $V[1 : r]$ as subsequences. It is obvious that $C[1 : l]$ is also an LCS of $A[1 : i]$ and $B[1 : j - 1]$ including $U[1 : k]$ and $V[1 : r]$ as subsequences.

The proof is now completed. $\square$

3 An Efficient Dynamic Programming Solution

Suppose the length of a longest common subsequence in $L(i, j, k, r)$ be denoted as $g(i, j, k, r)$. By Theorem 1 we know that the SEQ-IC-SEQ-IC-LCS problem will have the optimal substructure properties. Therefore, following recursive formula can be built to compute $g(i, j, k, r)$. For any $1 \leq i \leq n, 1 \leq j \leq m, 0 \leq k \leq s$, and $0 \leq r \leq t$, following recursive formula (1) can be used to compute the values of $g(i, j, k, r)$.

$$g(i, j, k, r) = \begin{cases} \max \{g(i - 1, j, k, r), g(i, j - 1, k, r)\} & \text{if } a_i \neq b_j \\ 1 + g(i - 1, j - 1, k - 1, r - 1) & \text{if } k > 0, r > 0, a_i = b_j = u_k = v_r \\ 1 + g(i - 1, j - 1, k - 1, r) & \text{if } k > 0, a_i = b_j = u_k \wedge (r = 0 \vee a_i \neq v_r) \\ 1 + g(i - 1, j - 1, k, r - 1) & \text{if } r > 0, a_i = b_j = v_r \wedge (k = 0 \vee a_i \neq u_k) \\ 1 + g(i - 1, j - 1, k, r) & \text{if } a_i = b_j \wedge (k = 0 \vee a_i \neq u_k) \wedge (r = 0 \vee a_i \neq v_r) \end{cases} \quad (1)$$

The boundary conditions to use this recursive formula read as follows.

$g(i, 0, 0, 0) = g(0, j, 0, 0) = 0$ and $g(i, 0, k, r) = g(0, j, k, r) = -\infty$ for any $0 \leq i \leq n, 0 \leq j \leq m, 0 \leq k \leq s$, and $0 \leq r \leq t$.

Our new algorithm to compute $g(i, j, k, r)$ is a standard dynamic programming algorithm based on above formula. The dynamic programming algorithm to compute $g(i, j, k, r)$ can be drawn up as the following Algorithm 1.

Algorithm 1. SEQ-IC-SEQ-IC-LCS

Input: Strings $A = a_1 \cdots a_n$, $B = b_1 \cdots b_m$ with their lengths n and m , respectively, and the two constrained sequences $U = u_1 u_2 \cdots u_s$ and $V = v_1 v_2 \cdots v_t$ with their lengths s and t

Output: $g(i, j, k, r)$, the length of a longest common subsequence of $A[1 : i]$ and $B[1 : j]$ including $U[1 : k]$ and $V[1 : r]$ as subsequences, for all $1 \leq i \leq n, 1 \leq j \leq m, 0 \leq k \leq s$, and $0 \leq r \leq t$.

```

1: for all  $i, j, k, r$ ,  $0 \leq i \leq n, 0 \leq j \leq m, 0 \leq k \leq s$  and  $0 \leq r \leq t$  do
2:    $g(i, 0, k, r), g(0, j, k, r) \leftarrow -\infty, g(i, 0, 0, 0), g(0, j, 0, 0) \leftarrow 0$  {boundary condition}
3: end for
4: for all  $i, j, k, r$ ,  $1 \leq i \leq n, 1 \leq j \leq m, 0 \leq k \leq s$  and  $0 \leq r \leq t$  do
5:   if  $a_i \neq b_j$  then
6:      $g(i, j, k, r) \leftarrow \max\{g(i-1, j, k, r), g(i, j-1, k, r)\}$ 
7:   else if  $k > 0$  and  $a_i = u_k$  then
8:     if  $r > 0$  and  $a_i = v_r$  then
9:        $g(i, j, k, r) \leftarrow 1 + g(i-1, j-1, k-1, r-1)$ 
10:    else
11:       $g(i, j, k, r) \leftarrow 1 + g(i-1, j-1, k-1, r)$ 
12:    end if
13:   else if  $r > 0$  and  $a_i = v_r$  then
14:      $g(i, j, k, r) \leftarrow 1 + g(i-1, j-1, k, r-1)$ 
15:   else
16:      $g(i, j, k, r) \leftarrow 1 + g(i-1, j-1, k, r)$ 
17:   end if
18: end for

```

It can be seen obviously that time and space complexities of the algorithm are both $O(nmst)$. The corresponding LCS of $A[1 : i]$ and $B[1 : j]$ including $U[1 : k]$ and $V[1 : r]$ as subsequences, for each value of $g(i, j, k, r)$ computed by algorithm *Suffix*, can be constructed by a backtracking algorithm pass through the paths from the computation (i, j, k, r) to $(0, 0, 0, 0)$. The following algorithm *back*(i, j, k, r) can be used as the backtracking algorithm to construct the longest common subsequence, not only its length. It is obvious that the time required by the algorithm *back*(i, j, k, r) must be $O(n + m)$.

4 Conclusions

We have proposed an efficient dynamic programming algorithm to solve the further generalized constrained LCS problem SEQ-IC-SEQ-IC-LCS. The worst case time complexity of the new dynamic programming algorithm is $O(nmst)$, where s, t, n, m are their lengths of the four given input sequences respectively. The space complexity of this problem can also be reduced to $O(\min(n, m)st)$, by using the technique based on Hirschbergs Algorithm [5].

There are many other generalized constrained LCS problems (GC-LCS) with similar structures of the problem dealt within this paper. It is not clear whether our technique developed in this paper can also be applied to these analogous

Algorithm 2. $back(i, j, k, r)$ **Input:** Integers i, j, k, r **Output:** The LCS of $A[1 : i]$ and $B[1 : j]$ including $U[1 : k]$ as a subsequence and $V[1 : r]$ as a suffix

```

1: if  $i < 1$  or  $j < 1$  then
2:   return
3: end if
4: if  $a_i \neq b_j$  then
5:   if  $g(i - 1, j, k, r) > g(i, j - 1, k, r)$  then
6:      $back(i - 1, j, k, r)$ 
7:   else
8:      $back(i, j - 1, k, r)$ 
9:   end if
10: else if  $k > 0$  and  $a_i = u_k$  then
11:   if  $r > 0$  and  $a_i = v_r$  then
12:      $back(i - 1, j - 1, k - 1, r - 1)$ 
13:   print  $a_i$ 
14:   else
15:      $back(i - 1, j - 1, k - 1, r)$ 
16:   print  $a_i$ 
17:   end if
18: else if  $r > 0$  and  $a_i = v_r$  then
19:    $back(i - 1, j - 1, k, r - 1)$ 
20:   print  $a_i$ 
21: else
22:    $back(i - 1, j - 1, k, r)$ 
23:   print  $a_i$ 
24: end if

```

problems to achieve the corresponding efficient algorithms. We will explore these problems further.

Acknowledgments. The work on this paper was supported by Data-Intensive Computing Key Laboratory of Fujian Province, and the Quanzhou Foundation of Science and Technology under Grant No.2013Z38.

References

1. Chen, Y.C., Chao, K.M.: On the generalized constrained longest common subsequence problems. *J. Comb. Optim.* **21**(3), 383–392 (2011)
2. Baeza-Yates, R.A.: Searching subsequences. *Theoret. Comput. Sci.* **78**, 363–376 (1991)
3. Deorowicz, S.: Quadratic-time algorithm for a string constrained LCS problem. *Inf. Process. Lett.* **112**(11), 423–426 (2012)
4. Deorowicz, S.: Bit-parallel algorithm for the constrained longest common subsequence problem. *Fundamenta Informaticae* **99**(4), 409–433 (2010)

5. Hirschberg, D.S.: Algorithms for the longest common subsequence problem. *J. ACM* **24**, 664–675 (1977)
6. Gotthilf, Z., Hermelin, D., Lewenstein, M.: Constrained LCS: hardness and approximation. In: Ferragina, P., Landau, G.M. (eds.) *CPM 2008*. LNCS, vol. 5029, pp. 255–262. Springer, Heidelberg (2008)
7. Gotthilf, Z., Hermelin, D., Landau, G.M., Lewenstein, M.: Restricted LCS. In: Chavez, E., Lonardi, S. (eds.) *SPIRE 2010*. LNCS, vol. 6393, pp. 250–257. Springer, Heidelberg (2010)
8. Masek, W.J., Paterson, M.S.: A faster algorithm computing string edit distances. *J. Comput. Syst. Sci.* **20**(1), 18–31 (1980)
9. Peng, Y.H., Yang, C.B., Huang, K.S., Tseng, K.T.: An algorithm and applications to sequence alignment with weighted constraints. *Int. J. Found. Comput. Sci.* **21**(1), 51–59 (2010)
10. Tang, C.Y., Lu, C.L.: Constrained multiple sequence alignment tool development and its application to RNase family alignment. *J. Bioinform. Comput. Biol.* **1**, 267–287 (2003)
11. Tsai, Y.T.: The constrained common subsequence problem. *Inf. Process. Lett.* **88**, 173–176 (2003)